

What is HTML

HTML is an acronym which stands for **Hyper Text Markup Language** which is used for creating web pages and web applications. Let's see what is meant by Hypertext Markup Language, and Web page.

Hyper Text: HyperText simply means "Text within Text." A text has a link within it, is a hypertext. Whenever you click on a link which brings you to a new webpage, you have clicked on a hypertext. HyperText is a way to link two or more web pages (HTML documents) with each other.

Markup language: A markup language is a computer language that is used to apply layout and formatting conventions to a text document. Markup language makes text more interactive and dynamic. It can turn text into images, tables, links, etc.

Web Page: A web page is a document which is commonly written in HTML and translated by a web browser. A web page can be identified by entering an URL. A Web page can be of the static or dynamic type. **With the help of HTML only, we can create static web pages.**

Hence, HTML is a markup language which is used for creating attractive web pages with the help of styling, and which looks in a nice format on a web browser. An HTML document is made of many HTML tags and each HTML tag contains different content.

Brief History of HTML

In the late 1980's , a physicist, Tim Berners-Lee who was a contractor at CERN, proposed a system for CERN researchers. In 1989, he wrote a memo proposing an internet based hypertext system.

Tim Berners-Lee is known as the father of HTML. The first available description of HTML was a document called "HTML Tags" proposed by Tim in late 1991. The latest version of HTML is HTML5, which we will learn later in this tutorial.

Tim Berners-Lee, a physicist at the CERN research institute in Switzerland invented HTML in 1991. This first version consisted of 18 HTML tags. Now, there are currently about 140 HTML tags, although not all of them are supported by modern browsers. Learn more about Tim Berners-Lee here:

[Tim Berners-Lee](#)



The Timeline of HTML

- **1991**- Tim Berners-Lee invents HTML 1.0
- **1993**- HTML 1.0 is released. Not many developers are creating websites at this time.
- **1995**- HTML 2.0 is published. This contains the features of HTML 1.0 plus new features. This remained the standard markup language for designing and creating websites until 1997.
- **1997**- HTML 3.0 was invented. Here, Dave Raggett introduced a fresh draft on HTML, which improved new features of HTML and gave more powerful characteristics for webmasters in designing websites. Unfortunately, the powerful features slowed down the browser in applying further improvements.
- **1999**- The widely-used HTML 4.0 comes out. It is very successful.
- **2014**- HTML 5.0 is released and used worldwide. It is said to be the extended version of HTML 4.01 which was published in 2012.

HTML Versions

Since the time HTML was invented there are lots of HTML versions in market, the brief introduction about the HTML version is given below:

HTML 1.0: The first version of HTML was 1.0, which was the barebones version of HTML language, and it was released in 1991.

HTML 2.0: This was the next version which was released in 1995, and it was standard language version for website design. HTML 2.0 was able to support extra features such as form-based file upload, form elements such as text box, option button, etc.

HTML 3.2: HTML 3.2 version was published by W3C in early 1997. This version was capable of creating tables and providing support for extra options for form elements. It can also support a web page with complex mathematical equations. It became an official standard for any browser till January 1997. Today it is practically supported by most of the browsers.

HTML 4.01: HTML 4.01 version was released on December 1999, and it is a very stable version of HTML language. This version is the current official standard, and it provides added support for stylesheets (CSS) and scripting ability for various multimedia elements.

HTML5 : HTML5 is the newest version of HyperText Markup language. The first draft of this version was announced in January 2008. There are two major organizations one is W3C (World Wide Web Consortium), and another one is WHATWG (Web Hypertext Application Technology Working Group) which are involved in the development of HTML 5 version, and still, it is under development.

Features of HTML

1) It is a very **easy and simple language**. It can be easily understood and modified.

2) It is very easy to make an **effective presentation** with HTML because it has a lot of formatting tags.

It is a **markup language**, so it provides a flexible way to design web pages along with the text.

4) It facilitates programmers to add a **link** on the web pages (by html anchor tag), so it enhances the interest of browsing of the user.

5) It is **platform-independent** because it can be displayed on any platform like Windows, Linux, and Macintosh, etc.

6) It facilitates the programmer to add **Graphics, Videos, and Sound** to the web pages which makes it more attractive and interactive.

7) HTML is a case-insensitive language, which means we can use tags either in lower-case or upper-case.

NOTE: It is recommended to write all tags in lower-case for consistency, readability, etc.

HTML Documents

- **HTML Documents:** HTML documents are structured using a specific format. A basic HTML document includes a `<!DOCTYPE html>` declaration, `<html>`, `<head>`, and `<body>` elements.
- **Using `<!DOCTYPE html>`:** This declaration defines the document type and version of HTML, helping browsers render the page correctly. It should be the first line in your HTML document.
- **The `<head>` Element:** Contains meta-information about the document, such as the title, character set, and links to stylesheets.
- **Using `<head>`:** Place metadata, links to CSS files, and other elements like `<title>`, `<meta>`, and `<link>` inside the `<head>` element.
- **The `<body>` Element:** Contains the content of the HTML document that is visible to users, including text, images, links, and other elements.
- **Using `id` and `class` Attributes:** The `id` attribute uniquely identifies an element, while the `class` attribute can be used to group multiple elements for styling or scripting.
-

HTML Text

- **Headings:** HTML provides six levels of headings (<h1> to <h6>), with <h1> being the highest level and <h6> the lowest.
- **Block and Inline Elements:** Block elements (e.g., <div>, <p>) start on a new line and take up the full width available, while inline elements (e.g., , <a>) flow within the text.

HTML Lists

- **List Types:** There are three main types of lists:
 - **Unordered Lists ():** Bulleted lists.
 - **Ordered Lists ():** Numbered lists.

Description Lists (<dl>): Lists of terms and descriptions.

- **Creating Lists:** Use elements for items within or , and <dt> and <dd> for description lists.
- **List Rendering:** Browsers render lists with default styling, but you can customize the appearance with CSS.

HTML Links

- **Link Concepts:** Links are created using the <a> element. The **href** attribute specifies the URL or location to which the link points.
- **Linking Documents:** Use the **href** attribute in the <a> tag to link to other pages or documents.
- **Understanding Targets:** The target attribute specifies where to open the linked document. Common values include **_blank (new tab)** and **_self (same frame)**.
-

HTML Tables

- **Table Elements:** Tables are structured using `<table>`, `<tr>` (table row), `<td>` (table data), and `<th>` (table header) elements.
- **Structuring a Table:** Define rows with `<tr>`, header cells with `<th>`, and data cells with `<td>`. Place header rows before data rows.
- **Table Data:** Populate the table with data using `<td>` inside `<tr>`.
- **Spanning Columns and Rows:** Use `colspan` and `rowspan` attributes in `<td>` or `<th>` to span multiple columns or rows.
- **Formatting Tables:** Customize table appearance with CSS, including borders, spacing, and alignment.

HTML Images and Objects

- **Image and Object Concepts:** Images are added using the `<img>` element, while other types of objects (**e.g., videos**) can be embedded with elements like `<object>`, `<embed>`, or `<video>`.
 - **Adding Images:** Use `` to insert images, where `src` is the image source and `alt` provides alternative text.
 - **Adding Objects:** Use the `<object>` element to include various types of content like multimedia or interactive elements.
-

HTML Documents

HTML Document Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>My First HTML Page</title>
  <meta charset="UTF-8">
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <h1>Welcome to My Website</h1>
  <p>This is a paragraph of text.</p>
</body>
</html>
```

Welcome to My Website

This is a paragraph of text.

HTML Text

Headings Example:

```
<h1>Main Heading</h1>
  <h2>Subheading Level 2</h2>
  <h3>Subheading Level 3</h3>

  <!-- Continue with h4, h5, h6 -->
```

Main Heading

Subheading Level 2

Subheading Level 3

Block and Inline Elements Example:

Block and Inline Elements in HTML

HTML elements are categorized as either **block-level** or **inline** based on how they behave within the document's layout.

Block-Level Elements

- Block-level elements start on a new line and take up the full width available (by default).
- They typically contain other block elements or inline elements.
- Common block-level elements include `<div>`, `<p>`, `<h1>` through `<h6>`, `<ul>`, `<ol>`, `<table>`, and `<section>`.

```
<div>
  <h1>This is a Heading</h1>
  <p>This is a paragraph of text within a block-
level element.</p>
  <ul>
    <li>List item 1</li>
    <li>List item 2</li>
  </ul>
</div>
```

This is a Heading

This is a paragraph of text within a block-level element.

- List item 1
- List item 2

Rendering:

- The `<h1>` starts on a new line and takes the full width.
- The `<p>` starts on a new line after the `<h1>`.

- The `<ul>` starts on a new line after the `<p>`, and its items (`<li>`) are rendered as part of the list structure.

Inline Elements

- Inline elements do not start on a new line; they appear within the flow of the text, side by side with other inline elements.
- They only take up as much width as needed by the content.
- Common inline elements include `<span>`, `<a>`, `<img>`, `<strong>`, `<em>`, and `<input>`.

```
<p>This is a paragraph with some <strong>bold  
text</strong> and an <a href="#">inline  
link</a>.</p>
```

This is a paragraph with some **bold text** and an [inline link](#).

Rendering:

- The `<strong>` element does not start on a new line; it simply makes the text bold within the flow of the paragraph.
- The `<a>` element is an inline link that appears within the same line as the rest of the text.

Combining Block and Inline Elements

Block-level elements can contain inline elements, but inline elements cannot contain block-level elements.

```
<div>  
  <p>This is a paragraph with a <a  
href="#">link</a> inside it.</p>  
  <span>This is an inline element, <strong>with  
bold text</strong>.</span>  
</div>
```

This is a paragraph with a [link](#) inside it.

This is an inline element, **with bold text**.

Rendering:

- The `<p>` element is block-level, so it starts on a new line.
- The `<a>` and `<strong>` elements are inline, so they are placed within the text flow.
- The `<span>` is inline and does not start on a new line, but stays within the flow of its container.

HTML Lists

List Types in HTML

HTML supports three main types of lists, each used for different purposes: **Unordered Lists**, **Ordered Lists**, and **Description Lists**. Each type serves a specific role in organizing content on a webpage.

1. Unordered List (`<ul>`)

- **Purpose:** To create a list where the order of items doesn't matter. Items are typically marked with bullets.
- **Tag:** `<ul>` (unordered list) contains `<li>` (list items).
- **Default Rendering:** Each item is preceded by a bullet point.

Unordered List:

```
<ul>
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
</ul>
```

```
<ul>
  <li>Apple</li>
  <li>Banana</li>
  <li>Cherry</li>
</ul>
```

Customization: You can change the bullet style using the list-style-type CSS property (**e.g., disc, circle, square, or none**).

```
<ul style="list-style-type: square;">
  <li>Apple</li>
  <li>Orange</li>
</ul>
```

Rendering:

- Apple
- Orange

. *Ordered List ()*

- **Purpose:** To create a list where the order of items is important. Items are numbered or lettered.
- **Tag:** **** (ordered list) contains **** (list items).
- **Default Rendering:** Each item is preceded by a number.

Ordered List:

```
<ol>
  <li>First item</li>
  <li>Second item</li>
  <li>Third item</li>
</ol>
```

Table Elements Example:

1. First item
2. Second item
3. Third item

Customization: You can change the numbering style using the list-style-type CSS property (e.g., **decimal**, **lower-alpha**, **upper-roman**).

```
<ol style="list-style-type: upper-roman;">
  <li>Apple</li>
  <li>Orange</li>
  <li>Mango</li>
</ol>
```

- I. Apple
- II. Orange
- III. Mango

Description List:

Description List (<dl>)

- **Purpose:** To create a list of terms and descriptions, such as definitions or pairs of items.
- **Tags:**
 - **<dl>** (description list) encloses the entire list.
 - **<dt>** (description term) specifies the term.
 - **<dd>** (description detail) specifies the description associated with the term.

```
<dl>
  <dt>HTML</dt>
```

```
<dd>HyperText Markup Language</dd>
<dt>CSS</dt>
<dd>Cascading Style Sheets</dd>
<dt>JavaScript</dt>
<dd>Programming language for the web</dd>
</dl>
```

HTML

HyperText Markup Language

CSS

Cascading Style Sheets

JavaScript

Programming language for the web

Nested Lists

You can also create nested lists, where a list item contains another list.

```
<ul>
  <li>Fruits
    <ul>
      <li>Apple</li>
      <li>Banana</li>
    </ul>
  </li>
  <li>Vegetables
    <ul>
      <li>Carrot</li>
      <li>Broccoli</li>
    </ul>
  </li>
</ul>
```

- Fruits
 - Apple
 - Banana
- Vegetables
 - Carrot
 - Broccoli

HTML Links

Link Concepts Example:

```
<a href="https://www.google.com">Google</a>
```

Linking Documents Example:

```
<a href="about.html">About Us</a>
```

Understanding Targets Example:

```
<a href="https://www.google.com" target="_blank">Open  
Google.com in a New Tab</a>
```

HTML Tables

Table Elements Example:

```
<table border=2>  
  <thead>  
    <tr>  
      <th>Header 1</th>  
      <th>Header 2</th>  
    </tr>  
  </thead>  
  <tbody>  
    <tr>  
      <td>Data 1</td>  
      <td>Data 2</td>  
    </tr>  
    <tr>  
      <td>Data 3</td>
```

```

        <td>Data 4</td>
      </tr>
    </tbody>
  </table>

```

Table Elements Example

Header 1	Header 2
Data 1	Data 2
Data 3	Data 4

Spanning Columns and Rows in HTML Tables

When working with HTML tables, you can span cells across multiple columns or rows using the `colspan` and `rowspan` attributes. This allows you to create more complex table structures, such as merging cells to form headers that span multiple columns or combining rows.

Spanning Columns with colspan

- **Purpose:** The `colspan` attribute is used to make a cell span across multiple columns.
- **Attribute:** `colspan="number"`, where number is the number of columns you want the cell to span.

Example: Spanning Columns

```

<table border="1">
  <tr>
    <th colspan="3">Header Spanning Three
Columns</th>
  </tr>
  <tr>
    <td>Row 1, Col 1</td>
    <td>Row 1, Col 2</td>
    <td>Row 1, Col 3</td>
  </tr>
  <tr>
    <td>Row 2, Col 1</td>

```

```

        <td colspan="2">Row 2, Col 2-3 (Spanned)</td>
    </tr>
</table>

```

Rendering:

Header Spanning Three Columns		
Row 1, Col 1	Row 1, Col 2	Row 1, Col 3
Row 2, Col 1	Row 2, Col 2-3 (Spanned)	

- The first row has a header cell (**<th>**) that spans three columns using **colspan="3"**.
- In the third row, the second cell spans two columns using **colspan="2"**.

Spanning Rows with rowspan

- **Purpose:** The **rowspan** attribute is used to make a cell span across multiple rows.
- **Attribute:** **rowspan="number"**, where number is the number of rows you want the cell to span.

Example: Spanning Rows

```

<table border="1">
  <tr>
    <th>Header 1</th>
    <th>Header 2</th>
  </tr>
  <tr>
    <td rowspan="2">Row 1-2, Col 1 (Spanned)</td>
    <td>Row 1, Col 2</td>
  </tr>
  <tr>
    <td>Row 2, Col 2</td>
  </tr>

```


`</table>`

Rendering:

Header 1	Header 2
Row 1-2, Col 1 (Spanned)	Row 1, Col 2
	Row 2, Col 2

Explanation:

- The first column of the second row spans two rows using **rowspan="2"**.
- The text in the spanned cell continues across both rows.

Combining colspan and rowspan

You can use both **colspan** and **rowspan** together to create cells that span multiple columns and rows simultaneously.

Example: Combining colspan and rowspan

```
<table border="1">
  <tr>
    <th colspan="2">Header Spanning Two
Columns</th>
    <th>Header 3</th>
  </tr>
  <tr>
    <td rowspan="2">Row 2-3, Col 1 (Spanned)</td>
    <td>Row 2, Col 2</td>
    <td>Row 2, Col 3</td>
  </tr>
  <tr>
    <td colspan="2">Row 3, Col 2-3 (Spanned)</td>
  </tr>
</table>
```

Rendering:

Header Spanning Two Columns		Header 3
Row 2-3, Col 1 (Spanned)	Row 2, Col 2	Row 2, Col 3
	Row 3, Col 2-3 (Spanned)	

Explanation:

- The first row's header spans two columns using **colspan="2"**.
- The first cell in the second row spans two rows using **rowspan="2"**.
- The third row's second cell spans two columns using **colspan="2"**.

Best Practices

- Use **colspan** and **rowspan** attributes to create complex table structures that are clear and well-organized.
- Ensure that the number of columns and rows after spanning is consistent with the table's overall structure to avoid layout issues.

This flexibility allows you to create tables that convey data more effectively, with headers that span across related columns and cells that merge vertically to represent shared data.

HTML Images and Objects

Adding Images Example:

```

```



Adding Objects Example:

```
<audio controls>
<source src="iphone.mp3" type="audio/mpeg">
</audio>
```

```
<video width="200" controls muted>

    <source src="sample.mp4" type="video/mp4">

</video>
```

```
<iframe width="560" height="315"
    src="//www.youtube.com/embed/YE7VzlLtp-4"
    frameborder="0"
    allowfullscreen></iframe>
```